Create PhoneApp.Conversations Context

Contexts are entirely opinion-based, so we're going to cover a technique that has worked well for me personally. This technique uses a combination of delegated functions and regular functions to create a context. Delegated functions live in another module (like our store modules defining database operations) and are supported out of the box in Elixir with defdelegate. Let's use it to create delegates for many of our store functions.

Create lib/phone_app/conversations/conversations.ex with the following code:

phone_app/lib/phone_app/conversations/conversations.ex

```
defmodule PhoneApp.Conversations do
  alias PhoneApp.Conversations.Query
  alias PhoneApp.Conversations.Schema

  defdelegate get_contact!(id), to: Query.ContactStore

  defdelegate create_sms_message(params), to: Query.SmsMessageStore
  defdelegate update_sms_message(sid, params), to: Query.SmsMessageStore
end
```

defdelegate/2 is an incredibly useful function. It defines a function on the module that calls a function in another module. (You can customize the function name if they don't match.) This lets us write our query code in an isolated module and then export it to the rest of the app in the context.

You may have noticed that this context doesn't include the data-loading functions that we wrote in our store. This is because we're going to improve on the API of these functions by wrapping them in a struct.

Create lib/phone_app/conversations/schema/conversation.ex with the following code:

phone_app/lib/phone_app/conversations/schema/conversation.ex

```
defmodule PhoneApp.Conversations.Schema.Conversation do
  @enforce_keys [:contact, :messages]
  defstruct @enforce_keys
end
```

This is a struct that requires contact and messages fields to be provided. This is added to the schema folder—even though it's not an Ecto schema—because it serves a role as part of our application's data structure.

Go back to the PhoneApp.Conversations context module and add these functions to the bottom:

phone_app/lib/phone_app/conversations/conversations.ex

```
def load_conversation_list do
  messages = Query.SmsMessageStore.load_message_list()
```